L3 informatique

Lecture Notes

Software Engineering

Practical Work
Requirements specification (IEEE 830 std)

Presented by
Votre noms.

Requirements Specification (cahier de charge)

- The term **"cahier des charges"** refers to a document that serves as a contract between two parties.

- This document (statement) describes the user's needs in terms of the functions to be provided and the objectives to be achieved.

- **Definition:** A document through which the requester expresses their need (or the need they are responsible for translating) in terms of service functions and constraints.

- The **requirements specification** is established during the initial phases (Phase 0) of the software life cycle, generally called the preliminary study or feasibility study

Requirement (besoin)

- A statement that translates or expresses a need and its associated constraints and conditions.
- **Categories of Needs**
  - **Functional Requirements**
    - Description of the services (functions).
    - Description of the data to be processed.
    - *“How the system is expected to be used.”*
  - **Non-Functional Requirements**
    - Description of the constraints.
    - For each service, as well as for the overall system, it is possible to express different types of constraints, such as:
      - Performance constraints
      - Security constraints
      - Usability and appearance constraints
      - Etc.

Writing a Requirements Specification

- **The requirements specification:**
  - Is written by the project owner (client), with the possibility of collaboration with the analyst, users, etc.
  - Uses natural language.
  - It may be written either:
    - in paragraphs clearly expressing the objectives, needs, and constraints,
    - or by using a standard, a norm, or a recommendation.

IEEE 830- 1998

- The term used by the standard is:

Software Requirements Specification (SRS).

- It should help:
  a) software purchasers to precisely describe what they are looking for;
  b) software suppliers to clearly understand what the client wants;
  c) individuals who aim to achieve the following objectives:
  - develop a standardized plan for a Software Requirements Specification (SRS) for their organization,
  - determine the format and content of their specific Software Requirements Specifications.

Advantages of a Well-Written SRS

- A well-written **Software Requirements Specification (SRS)** should provide several advantages, including:

  - Establishing the foundations of the agreement between the client and the supplier regarding the functions of the software.
  - Reducing development tasks.
  - Providing a basis for cost and schedule estimation.
  - Providing a baseline for validation and verification.
  - Facilitating transfer to new users or new machines.
  - Serving as a basis for upgrades or enhancements.

Definition of Terms Used

- **Contract:** A legally binding document agreed upon by the client and the supplier, which includes the technical and organizational requirements, the costs, and the schedule for the delivery of a product. The contract may also contain informal but useful information, such as the expectations or commitments of the parties.
- **Client:** A person or group who pays for the product and (usually) specifies its requirements. In the context of this document, the client and the supplier may belong to the same organization.
- **Supplier:** A person or group who delivers the product to the client. In the context of this document, the client and the supplier may belong to the same organization.
- **User:** A person who uses or interacts with the product.

Characteristics of a Well-Written SRS

- **Characteristics of a well-written Software Requirements Specification (SRS):**
  a) **Correct**
  b) **Unambiguous**
  c) **Complete**
  d) **Consistent**
  e) **Ranked for importance** *(or Categorized)*
  f) **Verifiable**
  g) **Modifiable**
  h) **Traceable**

Characteristics of a Well-Written SRS

- **Correct**
  An SRS is correct if and only if every requirement it describes must be satisfied by the software.
- **Unambiguous**
  An SRS is unambiguous if and only if every requirement it describes has a single interpretation (natural language).
- **Complete**
  An SRS is complete if and only if it contains all of the following elements:
  - All significant requirements,
  - The software's response to all classes of inputs in all possible situations,
  - The definition of all terms, and the presence of all legends and references to figures and tables.
- **Consistent**
  An SRS is consistent if and only if no subset of requirements conflicts with another.

Characteristics of a Well-Written SRS

- **Ranked/Categorized**
  Each requirement must have a value that places it in a certain position within a category (essential, conditional or optional)
- **Verifiable**
  An SRS is verifiable if and only if every requirement it contains can be objectively verified.
- **Modifiable**
  An SRS is modifiable if and only if its structure and writing style are such that each change to the requirements is easy to make and keeps the document consistent without altering its structure and style.
- **Traceable**
  An SRS is traceable if the origin of each requirement is clear, and if it allows integration of references to the evolution of the requirements and to the artifacts that elaborate them.

What an SRS Should Not Do

- Organize the software into “modules”
- Assign functions to modules
- Show the flow of control (with caution!)
- Define the data structures

The Parts of the

- **Table of Contents**
- **1. Introduction**
  1.1 Purpose
  1.2 Scope
  1.3 Definitions, Acronyms, and Abbreviations
  1.4 References
  1.5 Overview
- **2. General Description**
  2.1 Environment
  2.2 Functions
  2.3 User Characteristics
  2.4 Constraints
  2.5 Assumptions and Dependencies
- **3. Specific Requirements**
  (*Note: several alternative organizations are possible for this section.*)
- **Appendices**
  Index

Detailed Sections of a SRS

- **Purpose (Section 1.1 of the SRS)**
  This section should accomplish the following tasks:
  a) State the purpose of the SRS,
  b) Specify the intended audience of the SRS.
- **Scope (Section 1.2 of the SRS)**
  This section should:
  a) Identify by name the software product(s) to be produced (e.g., database management system, report generator, etc.),
  b) Explain what the software will and, if necessary, will not do,
  c) Describe the applications of the specified software, including relevant benefits, objectives, and goals,
  d) Be consistent with similar statements in higher-level specifications (e.g., system requirements specifications), if applicable.
- **Definitions, Acronyms, and Abbreviations (Section 1.3 of the SRS)**
  This section should provide the definition of all terms, acronyms, and abbreviations that the reader may need in order to properly interpret the SRS. This information may be supplied in appendices to the SRS or by reference to other documents.

Detailed Sections of a SRS

- **References (Section 1.4 of the SRS)**
  This section should:
  a) Provide a complete list of all documents referenced in the SRS,
  b) Give the title, number (if applicable), date of publication, and publisher of each document,
  c) Specify the sources from which these documents can be obtained.
- **Overview (Section 1.5 of the SRS)**
  This section should:
  a) Describe the remaining sections of the SRS,
  b) Explain how the SRS is organized.

Detailed Sections of a SRS

- **General Description (Chapter 2 of the SRS)**
  This chapter of the SRS should describe the general factors that influence the product and its requirements.
- **Environment (Section 2.1 of the SRS)**
  This section should place the product in the context of related products. If it is independent and fully autonomous, this must be stated here.
  This section should also indicate the constraints to which the software must conform, including:
  a) Interfaces with the system,
  b) Interfaces with users,
  c) Interfaces with hardware,
  d) Interfaces with software,
  e) Communication interfaces, etc.
- **Functions (Section 2.2 of the SRS)**
  This section provides a summary of the main functions that the software must perform.
  For the sake of clarity:
  a) The functions should be organized in such a way that the list can be easily understood by the client or anyone reading the document for the first time.
  b) The different functions and their relationships may be presented in either textual or graphical form. Such a diagram is not intended to show the product's design but simply to indicate the logical relationships among various elements.
- **User Characteristics (Section 2.3 of the SRS)**
  This section should describe the general characteristics of the product's users, including: level of education, experience, and technical knowledge.

Detailed Sections of a SRS

- **Constraints (Section 2.4 of the SRS)**
  This section of the SRS provides a general description of any other factors that may limit the options available to the designer, including:
  a) Regulatory policies,
  b) Hardware limitations (e.g., signal timing requirements),
  c) Interfaces with other applications,
  d) Parallel operation,
  e) Verification functions,
  f) Control functions,
  g) Requirements related to high-level languages, etc.
- **Assumptions and Dependencies (Section 2.5 of the SRS)**
  This section of the SRS lists all factors that influence the requirements stated in the document. It does not address design constraints, but rather potential changes to them that may have an impact on the requirements.

Detailed Sections of a SRS

- **3. Specific Requirements**
  (*There are eight possible organizations; here we adopt the one organized by mode.*)
- **3.1 Functional Requirements**
- **3.1.1 Mode 1**
  - **3.1.1.1 External Interface Requirements**
    - **3.1.1.1.1 User Interfaces**
    - **3.1.1.1.2 Hardware Interfaces**
    - **3.1.1.1.3 Software Interfaces**
    - **3.1.1.1.4 Communication Interfaces**
  - **3.1.1.2 Functional Requirements**
    - **3.1.1.2.1 Functional Requirement 1**
    - …
    - **3.1.1.2.n Functional Requirement 1.n**
  - **3.1.1.3 Performance Requirements**
- **3.1.2 Mode 2**
  - …
- **3.1.m Mode m**
- **3.2 Performance Requirements**
  3.3 Design Constraints
  3.4 Attributes
  3.5 Other Requirements

Detailed Sections of a SRS

- **Mode**
  To organize the chapter by mode, use structures such as: User Classes, Objects, Characteristics, Function Hierarchy, etc.
- **Functional Requirements**
  Functional requirements define the main actions that the software must perform, including the reception and processing of inputs, as well as the processing and generation of outputs.
- **Performance Requirements**
  This section specifies the quantitative requirements—both static and dynamic—that must be satisfied by the software or by the interaction between the human and the software. Static requirements may include:
  a) The number of terminals it must support,
  b) The number of users it must support simultaneously, etc.
- **Design Constraints**
  This section specifies the design constraints that may be imposed by other standards, hardware limitations, etc.
- **Attributes**
  Certain software attributes may serve as requirements. It is important to specify the required attributes so that their implementation can be objectively verified. A partial list includes: Availability, Security, Maintainability, Portability, etc.